



GOPS 2026
Beijing

智能体驱动的

第29届

GOPS

2026

全球运维大会



暨研运数智化技术峰会

2026年7月10日-11日

中国·北京

指导单位：



主办单位：



承办单位：



借力 AI RCA 筑牢企业生产稳定性实战



2026.7.10

姓名

秦晓辉

秦晓辉



快猫星云 · 联合创始人

- 两代开源监控系统（Open-Falcon、夜莺监控）构建者
- 极客时间专栏《运维监控系统实战笔记》作者
- 公众号 SRETALK 主理人
- 可观测性方向 ToB 创业者



类似 PagerDuty 的 On-call 平台，
东半球最好用的 On-call 产品



夜莺监控的商业版，可观测性平台，
稳定性治理平台



目录

CONTENT

01

业内实践与 AI RCA 整体架构

02

RCA Layer1: Incident 层的 AI Agent 实践

03

RCA Layer2: O11y 层的 AI Agent 实践

04

RCA Layer3: 实时诊断层的 AI Agent 实践

01 业内实践与 AI RCA 整体架构

调研数十家头部可观测性厂商，大家用 AI 强化原本就擅长的东西，工程和数据积累才是根基



Dynatrace

Grail 统一数据底座 + Smartscape 实时依赖图 + Causal AI 因果判断



Datadog

Bits AI SRE 生成假设 → 调用 telemetry → 验证或推翻



PagerDuty / Incident.io

告警归一化为 Incident，作为排障入口；深入 Slack 工作流作为智能助手



Resolve AI

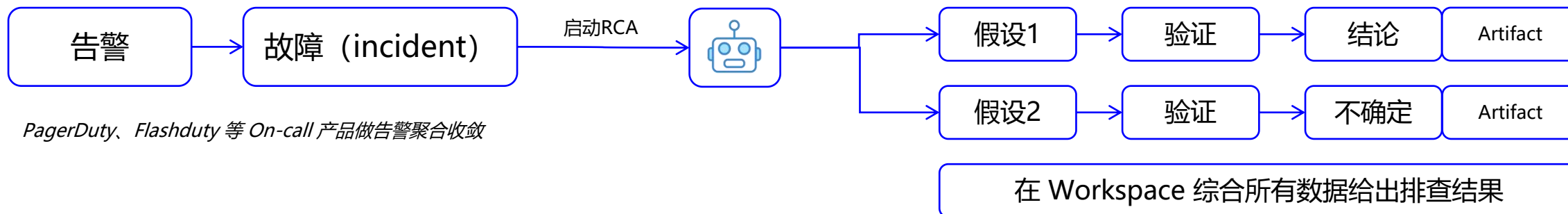
RCA 新贵，不持有可观测性数据也不持有 Incident，串联各类系统把 RCA 产品化

核心发现：给 AI 地图、元信息和工具，AI 才能更好地工作 —— 而不是让 AI 自由探索



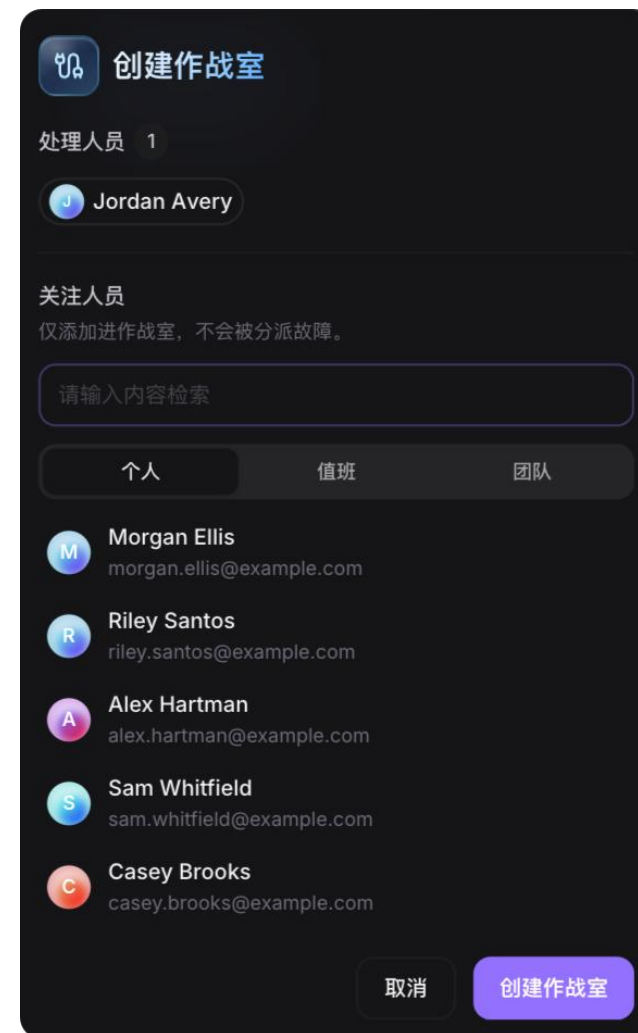
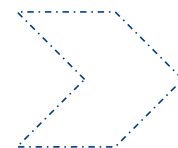
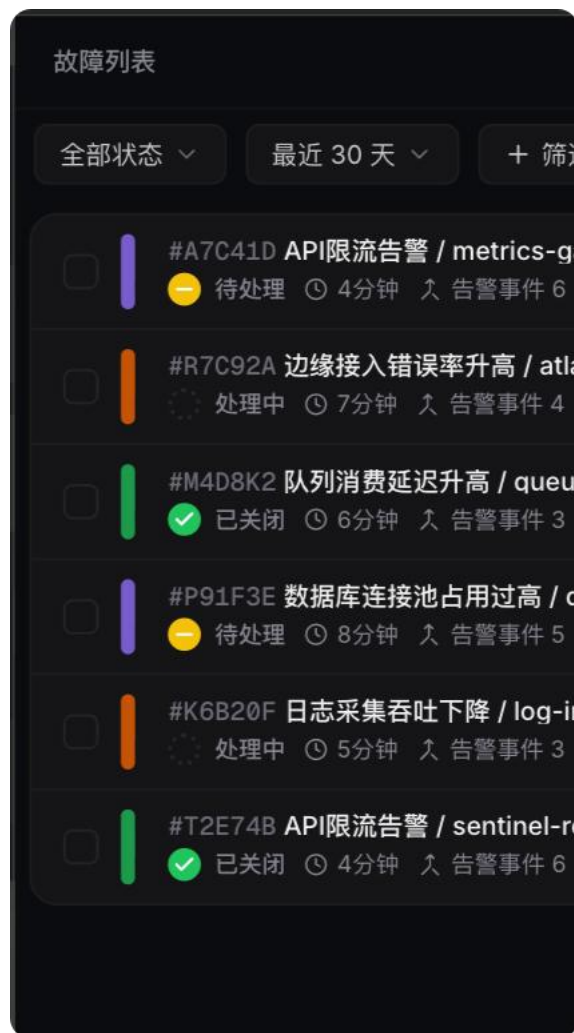
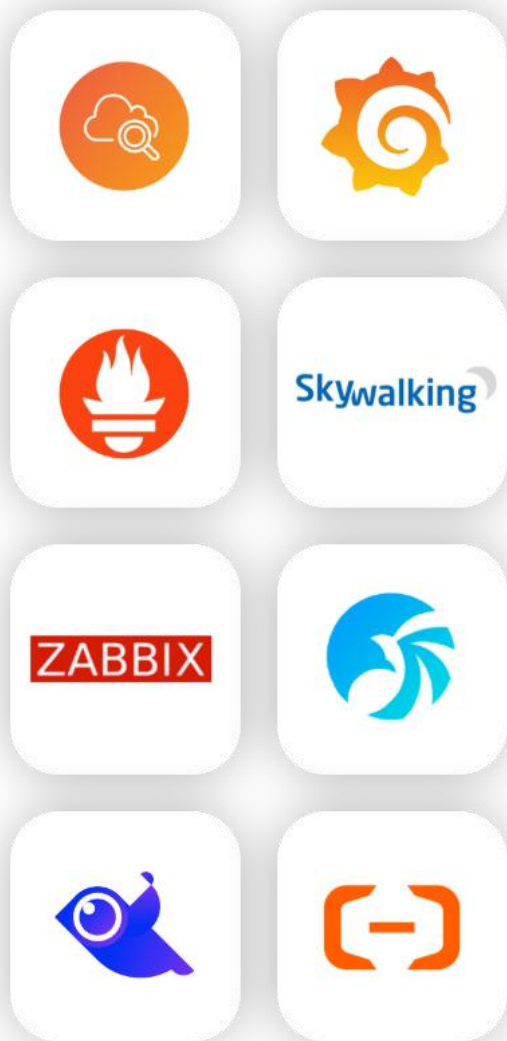
AI RCA 并不是引入 AI 之后让 AI 自由探索（准确性太差），而是给 AI 提供地图、元信息和工具，让 AI 在受控环境中高效工作。

业内 AI RCA 整体架构



02 RCA Layer1: Incident 层的 AI Agent 实践

Incident 层是怎样一个东西



核心治理职能

⚡ 告警收敛

避免噪音、避免重复 RCA

≡ 归一化

语义和结构对齐 (Severity、状态等)

🔍 富化

附加更多属性和标签，丰富上下文

🗨️ 统一排障入口

共享协同的排障工作台

响应协作职能

围绕 Incident 的完整响应协作：

- 排班：责任到人，不打扰其他人
- 认领、升级：确保告警必须有人跟进
- 静默、抑制、合并：智能降噪
- 自动拉群、通知：协作自动化
- 自动总结、更新 Statuspage
- 复盘沉淀：跟进结果可分析、可运营



Incident 历史复盘记录和相似 Incident 是 AI RCA 的重要参考，可周期性整理为 Memory

Incident 层的哪些功能可以引入 AI

On-call 产品主战场（国外 PagerDuty、国内 Flashduty） — AI 是助手，不是替代 workflow

告警聚合

两种方式并行：

- 向量库相似度：不用写规则，可解释性差
- 基于规则：可解释性强，更麻烦

实际使用：50-50

自动复盘

自动生成复盘记录：

- 读取 Incident 信息
- 读取时间线
- 读取飞书/Slack 历史消息

Assistant 能力

- 辅助编辑消息模板
- 问答机器人
- 根据代码生成文档
- 读取文档回答问题



关键洞察

On-call 本质解决的是协作问题，是人类参与的工作流。AI 的定位是助手，而非替代工作流的环节。

1 天然的共享协同入口

Incident 是团队共同面向的 workspace，大家共享 Skills、Memory、历史复盘记录，方便协同排障。工程师的 Cursor、Claude 是个人入口

2 结构化、信息丰富的对象

"帮我诊断 Incident: 6a190e5e" 比 "帮我分析 xx 服务 Latency 升高的问题" 更方便 AI 获取结构化信息，比如故障时间、标签等。

3 天然会对接各类数据源

On-call 类产品天然会对接各类数据源，因为要统一告警，这些“集成关系”天然可以复用到 RCA 场景，以 Flashduty 举例，对接了数十种数据源

03 RCA Layer2: O11y 层的 AI Agent 实践

指标 Metrics

- Prometheus
- VictoriaMetrics
- Thanos

问题：通常有多套并存

日志 Logs

- Elasticsearch
- Loki / VictoriaLogs
- Doris / SLS / ClickHouse

问题：半结构化，大部分研发仅仅用来 grep 关键字，没有把统计分析能力用起来

链路 Traces

- SkyWalking / SigNoz
- Jaeger / Tempo / Zipkin
- Pinpoint / ARMS

问题：通常只有部分服务对接

现实问题

数据完备性、语义一致性、告警规范性、信息层级性都很差，需要长时间治理；而且数据通常分散在多种平台，RCA 获取数据时略麻烦。

开放问题：这些数据是否一定要统一存储？

O11y 数据是否应该统一存储（Honeycomb、CK 派系）？

统一更好，不统一问题也不大 —— 本质只是查询参数不同

不同数据源查询示例

```
/query?datasource=prom-01&q1=promql
```

```
/query?datasource=loki-01&q1=logql
```

```
/query?q1=dsl-01
```

```
/query?q1=dsl-02
```

关键点：

- 不同数据源虽然采用不同 QL，但这不是太大问题
- LLM 已经内化了这些查询语言，甚至比自己发明的统一QL更好
- 除非确实需要 join 查询，这个场景比较少

! 真正关键的问题

数据语义规范一致比存储统一更重要。指标里有个 payment 的 service，日志里也得有个 payment 的 index，如果日志里叫 pay，那就需要额外的 mapping knowledge。



→ AI Agent 对 O11y 数据有哪些核心诉求？

1 索引信息

从 Incident 字段中找到"索引"信息

如 **service**、**instance** 等告警字段，根据这些字段找到关键 O11y 数据

Incident → O11y 数据

2 "总-分"访问路径

过滤到 service、instance 后仍有很多数据

需要"总-分"结构：

- 全局 Overview 视图到具体条目
- log pattern 查询
- 先看关键 SLI / 仪表盘，再看具体指标

3 关联对象数据

除了告警对象自身数据，还需要：

- 依赖的 Runtime 环境，机器、基础网络等
- 依赖的其他自研的微服务
- 各类通用的中间件、数据库、MQ 等
- 第三方服务接口，比如支付宝的接口

🌊 最难搞的深水区

数据关联是最大挑战 —— 生产环境就是语义不规范的，有些数据就是不全的：Tracing 里没有 Runtime 依赖，甚至有些服务就是没对接 Tracing。怎么办？

→ 接下来看看头部厂商如何处理这些挑战

Dynatrace

为 IT 世界建模

- Grail 统一数据底座
- Smartscape 实时依赖图
- Causal AI 因果判断

前提：OneAgent 统一采集 → 统一规范 → 统一拓扑

问题：真的可以 100% 覆盖吗？

Datadog

探索式"假设-验证"

- 多 thread 并行验证
- 依据：Incident 标签、Tracing、Memory

承认生产环境"乱七八糟"的特性现状

更务实、更灵活

Grafana

比较克制、保守

- 优先做 Assistant
- Sift 自动化排障 checklist

强调辅助调查，调查过程可视化、可信赖

比给出一个错误结论更有用

核心差异：承认现实 vs 理想建模 —— 生产环境就是"乱七八糟"的，需要持续治理的

洞察：数据是海量的，Entity 是可枚举的

海量的数据：指标、日志、Traces、Events 都是海量的

可枚举的 Entity：

- 微服务、数据库实例
- 专线、业务功能

需要在 **Entity 层级** 建设 O11y RCA 模型

Entity 的关键特征

- ✓ 有健康状态
- ✓ 有 SLI 指标
- ✓ 可能关联日志/Tracing
- ✓ 健康与否可被量化

Entity 是大部分排查入口，决定 Entity 健康的因素方向可枚举，这为 AI RCA 奠定了基础

 **流量** (比如突增)

 **依赖** (比如其他服务异常)

 **变更** (比如配置变更)

"流量"和"变更"可看作 Entity 属性，"依赖"需要从 Tracing、K8s 生成或人工配置

■ 灭火图：全域 Entity 健康视图、驾驶舱

核心设计：

- Entity 关联数据，告警与 Entity 绑定
- Entity 多个量化指标归到 Status
- 本质是在 Entity 层面做告警聚合
- Entity 分类、分层级，底层故障上浮
- Entity 构建基于稳定规则
- 规则提取让 AI 完成，既高效又不漂移



为什么叫"灭火图"？

就像消防员的灭火指挥图 —— 一眼看清全域态势，哪里"着火"（异常）、火势多大（严重程度）、应该先去救哪里（优先级），全部一目了然。

功能接口 4



微服务 2



中间件 3



基础设施 4



■ 灭火图核心设计思考，面向 RCA 的知识图谱入口

把海量可观测性数据提取关键骨架，让人类和 AI 都更容易理解和使用



对象上下文

接口、服务、数据库
缓存、主机、业务系统



健康上下文

成功率、延迟、错误率
慢查询、堆积、可用性



关系上下文

依赖关系、归属关系
调用链路、部署拓扑



数据路径上下文

日志、Trace、仪表盘
事件、变更的下钻入口

灭火图 = 对象上下文 + 健康上下文 + 关系上下文 + 数据路径上下文

AI 需要的是上下文，灭火图提供的是完整的 RCA 上下文体系

不是只输出"可能是数据库问题"——可信度来自证据链，不来自措辞自信

- | | |
|----------------------------------|------------------------------------|
| 1 事实
哪些对象异常，影响哪个业务 | 4 证据
指标、日志、Trace、事件、拓扑链接 |
| 2 时间线
异常从什么时候开始，是否有变更 | 5 反证
哪些方向已经排除 |
| 3 候选根因
按置信度排序，拿不准就写拿不准 | 6 下一步
对哪个对象做进一步验证 |

如果硬要 AI 输出结论，AI 就会过度归因
拿不准的就写拿不准 —— 诚实比自信更重要

→ 可观测性层能形成候选根因和证据链，但有些问题必须进入第三层做现场确认

04 RCA Layer3: 实时诊断 层的 AI Agent 实践

很多数据不全，有些数据不会上报到可观测性系统 —— 必须到现场确认



句柄快用满了 —— 文件句柄还是网络句柄？哪个进程占用？



磁盘满了 —— 具体哪个子目录？哪些大文件导致？



MySQL 锁等待 —— 什么原因导致？



ES Health Yellow —— 哪个索引分配失败？



Redis 延迟 —— 大 Key？硬件资源？具体是哪个 Key？

➤ 不可避免，仍然需要 ssh 到机器上，仍然需要连到数据库上才能诊断，O11y 总感觉差点啥

- 1 所有机器部署 agent
执行只读诊断能力，获取机器实时信息
- 2 安全机制
shell 不够安全 → 白名单 AST 解析和限制；sandbox 更安全但限制太大，效果打折扣
- 3 结构化 Tool
如 os.overview、http.get 等 —— 提高安全性，减少 LLM 交互轮次
- 4 数据库/MQ 诊断
由持有认证信息的 agent 查询诊断；监控采集器可改造复用（已持有认证信息）
- 5 分布式路由
agent 上报 target 到中心，中心提供分布式路由；LLM 调用统一 invoke 接口，websocket 穿透到 agent

架构：AI Agent → 中心 invoke 接口 → websocket → 本地 Agent → 结构化 Tool → Target

一个 Tool = 对某个 target 的一次受控观察

硬约束

只读

轻量 (秒级)

同步返回

超时 + 截断 + 脱敏

更多约束

- 有审计记录
- 凭据留在本地 Agent
- 不进入平台侧日志和路由
- 静态内置能力
- 不默认远端下发动态脚本

? 一个开放问题

未来是否应该开放远端下发动态脚本的能力?

一旦开放:

- 灵活性大幅提升 ↑
- 攻击面也大幅扩大 ↑

Tool 分层: snapshot → focused → flexible

一个 Tool 应该回答一个明确的 SRE 诊断问题

snapshot

排障起手，快速判断方向

示例:

- os.overview
- mysql.overview
- redis.overview

快速获取全局状态

focused

验证具体假设

示例:

- mysql.lock_contention
- redis.slowlog

针对特定问题的深入诊断

flexible

受控探索长尾问题

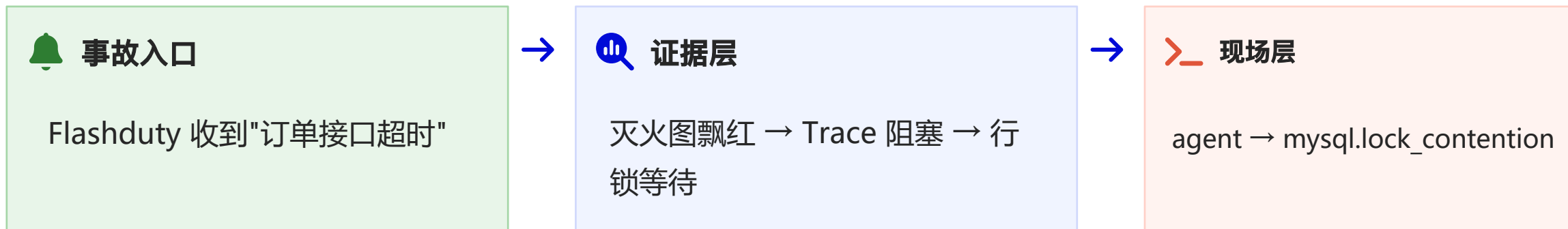
示例:

- shell.exec (AST 解析, 白名单)
- mysql.query (SQL Parser)

处理非标准场景的灵活性

不要做一个 diagnose_mysql 大而全工具，也不要复制底层系统完整 API —— 颗粒度太粗 AI 不知道重点，太细调用链过长

示例：MySQL 死锁 / 锁等待 RCA



完整诊断流程

1. Flashduty 收到 incident: "订单接口超时"
2. Flashcat 灭火图: 订单服务和订单库飘红
3. Trace: 请求阻塞在数据库调用
4. MySQL 大盘: 线程运行数和行锁等待上升
5. Agent 调用: mysql.lock_contention

最终输出

- 谁被阻塞
- 谁是 blocker
- 哪类 SQL / 事务相关

建议: kill 会话 / 回滚 / 限流 / 观察

01 Incident 是 RCA 的入口

Flashduty 这类 On-call 产品解决告警收敛、治理、响应协作和复盘沉淀，串联各类 O11y 数据源和其他结构化数据源

02 可观测性是 RCA 的证据层

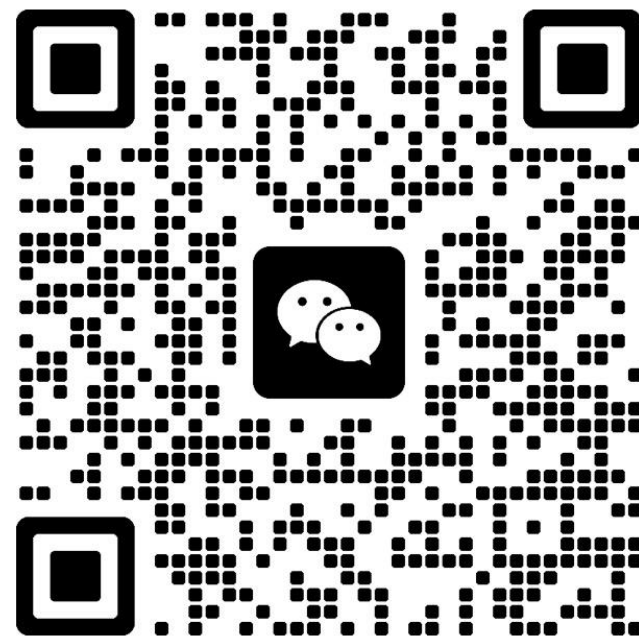
Flashcat 这类可观测性产品用灭火图和全栈数据构建 AI-Ready 上下文，让 AI RCA 有地图有路径

03 实时诊断是 RCA 的现场层

用只读、受控、可审计的 Tool 帮 AI 验证假设，曾经做过一个 Demo: github.com/cprobe/catpaw 已过时仅供参考



秦晓辉
中国大陆



扫二维码，添加我为朋友。



Thanks



高效运维社区
BizDevOps 软件工厂 荣誉出品



GOPS 2026
Beijing

智能体驱动的

第29届

GOPS

2026

全球运维大会



暨研运数智化技术峰会

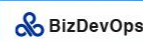
2026年7月10日-11日

中国·北京

指导单位：



主办单位：



承办单位：

